



ARQUITETURA E DESENVOLVIMENTO JAVA

FASE 5 – SUBSTITUTIVA

Tech Challenge

O Tech Challenge é o projeto de fase que englobará os conhecimentos obtidos em todas as disciplinas. É importante atentar-se ao prazo de entrega, pois esta é uma atividade obrigatória, que vale 60% da nota de todas as disciplinas da fase.

Objetivo:

Desenvolver um sistema de gerenciamento de encomendas para prédios residenciais, por meio do qual porteiros recebam e organizem as entregas, utilizando tecnologias para que o sistema notifique moradores sobre a chegada das encomendas e para que eles confirmem seu recebimento. Além disso, esse sistema deve registrar automaticamente a retirada das encomendas na portaria. O projeto deve ser configurado para ser executado em um contêiner Docker, seguindo os princípios da Arquitetura Limpa.

Funcionalidades:

O sistema deve apresentar as seguintes funcionalidades:

1. Cadastro de moradores:

- a. Será necessário criar uma API que permita a moradores a se cadastrarem ou alterarem suas informações. As principais informações são telefone e apartamento.
- b. Moradores devem ter um login baseado em OAuth2 ou JWT, de acordo com a escolha do grupo.
- c. A função de cadastro pode ser feita sem estar logado, porém, para qualquer outra função, a pessoa deve estar logada antes de realizar uma alteração dos dados.

2. Cadastro de funcionários:

- a. Todos os funcionários devem ser registrados com Login e Senha para poder utilizar o sistema.
- b. A tecnologia de autorização deve seguir o padrão definido para o Morador.
- c. A função de cadastro pode ser feita sem estar logado, porém para realizar qualquer outra função a pessoa deve estar logada antes de efetuar uma alteração dos dados.

3. Recebimento de encomendas pela portaria:

- a. Porteiros podem receber encomendas de entregadores, registrando informações como nome do morador destinatário, número do apartamento e descrição da encomenda.

- b. As encomendas recebidas devem ser colocadas em uma fila de processamento.
- c. Essa função só pode ser acessada pelos porteiros.
- 4. Notificação aos moradores:**
 - a. O sistema deve notificar moradores sobre a chegada das encomendas através de mensagens enviadas para os seus dispositivos (por exemplo, aplicativo móvel, e-mail, SMS).
 - b. Os moradores podem confirmar o recebimento da mensagem de notificação logando no sistema com o seu usuário e senha.
- 5. Baixa das encomendas retiradas:**
 - a. Quando um morador retirar uma encomenda na portaria, o porteiro deve dar baixa no sistema, registrando a retirada da encomenda pelo morador.
 - b. Essa função deve ser acessada somente pelo porteiro.
- 6. Integração com mensageria:**
 - a. As encomendas recebidas pelo porteiro devem ser enviadas para um canal de entrada.
 - b. As mensagens de notificação aos moradores devem ser enviadas para um canal de saída.
- 7. Persistência de dados:**
 - a. As informações das encomendas recebidas e das notificações enviadas aos moradores devem ser armazenadas em um banco de dados para posterior consulta e análise.
- 8. Testes Unitários e de Integração:**
 - a. O sistema deve ser testado realizando testes unitários para garantir que classes e métodos individuais funcionem corretamente.
 - b. Testes de integração devem ser realizados para garantir que o sistema funcione.
- 9. Execução em Docker:**
 - a. O projeto deve ser configurado para ser executado em um contêiner Docker.
 - b. Deve ser fornecido um Dockerfile para construir a imagem do contêiner.

Entregáveis:

Você deverá apresentar um PDF contendo:

- **Link do GitHub:** deve ser fornecido o link do repositório GitHub com o código-fonte completo do projeto.
- **Documentação técnica:** documentação detalhada do sistema e da API, utilizando ferramentas como Javadoc ou Swagger.

- **Relatório técnico:** uma descrição das tecnologias e ferramentas utilizadas, os desafios técnicos enfrentados e as soluções adotadas, com ênfase na aplicação dos princípios de Clean Architecture e qualidade de software.

Avaliação:

1. Clean Architecture:

- a. **Design Arquitetônico:** aplicar a Clean Architecture no backend, definindo claramente as entidades, casos de uso, adaptadores e frameworks externos.

2. Qualidade de Software:

- a. **Testes Unitários (TDD):** utilizar Test-Driven Development (TDD) para criar o backend, buscando alta cobertura de testes com JUnit ou outras ferramentas adequadas.
- b. **Testes Integrados e Análise de Código:** realizar testes que integram diferentes módulos do sistema para verificar a interação entre eles e utilizar ferramentas de análise de código estático para garantir a qualidade.
- c. **Execução dos testes:** lembre-se de deixar as linhas de código para executar cada um dos testes listados acima.